TabTank — Updated Build Plan (v4)

Turn your new tab into an aquarium. Draw your own fish and watch them swim.

Complete Build Plan — Chrome Extension

Table of Contents

- What You Already Have
- What Needs to Change
- Phase 1 — Skeleton (Get Something On Screen)
- Phase 2 — Drawing Tool (painter.js)
- Phase 3 — Fish Spawning & Swimming (fish.js)
- Phase 4 — Ocean Backgrounds & Mode System
- Phase 5 — Bait Feature
- Phase 6 — Persistence (storage.js)
- Phase 7 — Bubbles (bubbles.js) — Optional Polish
- Phase 8 — Custom UI & Button Art
- Layer Stack & Z-Index Order
- Milestone Checklist

What You Already Have

manifest.json (Complete)

Manifest V3 Chrome extension that overrides the new tab page with `newtab.html`. Permissions for `storage` and `unlimitedStorage` are already declared. Icons referenced at 16, 48, and 128px sizes.

```
{
  "manifest_version": 3,
  "name": "TabTank",
  "version": "1.0.0",
  "description": "Turn your new tab into an aquarium. Draw your own fish and wat
  "chrome_url_overrides": {
    "newtab": "newtab.html"
  },
  "permissions": ["storage", "unlimitedStorage"],
  "icons": {
    "16": "icons/icon16.png",
    "48": "icons/icon48.png",
    "128": "icons/icon128.png"
  }
}
```

Git Repository Initialized

newtab.html (Complete)

Fullscreen overlay design with floating draw button, toolbar, and large draw canvas.

css/style.css (Complete)

Layout rules for body, ocean, fish canvas, draw button, draw overlay, toolbar, draw canvas, and bait bucket.

js/painter.js (In Progress)

Overlay toggle, freehand drawing, color picker, brush size, eraser, and clear button implemented.

What's Still Missing

Fish spawning and swimming (fish.js), persistence (storage.js), bubbles (bubbles.js), background art, mode system, custom button art.

What Needs to Change

The drawing experience was redesigned from v1. Instead of a small draw panel always visible in the bottom-right corner, the new design uses:

1. **A floating "Draw" button** — a single `+` button visible on the aquarium screen
2. **A fullscreen drawing overlay** — clicking the button opens/closes a fullscreen view where the entire page becomes the drawing canvas with a toolbar

This is a better UX because: - More drawing space — you're not cramped into a 300×200 box - Cleaner aquarium view — no panel cluttering the screen - Room for new features — fish naming, larger brush preview, etc.

Phase 1 — Skeleton (Get Something On Screen)

Status: COMPLETE

Step 1.1: File Structure

```
TabTank/
├── manifest.json              ✅ exists
├── newtab.html                ✅ exists
├── css/
│   └── style.css              ✅ exists
├── js/
│   ├── painter.js             ⚠️ in progress
│   ├── fish.js                ⚠️ in progress
│   ├── bubbles.js
│   └── storage.js
├── assets/
│   ├── backgrounds/
│   │   ├── ocean/             (standard ocean art)
│   │   └── pixel/             (pixel art backgrounds, 320×180)
```

```
│   └── ui/                     (custom button art)
└── icons/
    ├── icon16.png              ✅ exists
    ├── icon48.png              ✅ exists
    └── icon128.png             ✅ exists
```

Step 1.2: newtab.html

Aquarium view (always visible): - `<div id="ocean">` — background container for your art - `<canvas id="fishCanvas">` — fullscreen transparent fish swimming surface - `<button id="drawBtn">+</button>` — floating button that toggles the drawing overlay - `<div id="baitBucket">` — bait feature element

Drawing overlay (hidden by default, toggled by drawBtn): - `<div id="drawOverlay">` — fullscreen container, hidden with `display: none` - `<div id="toolbar">` — horizontal bar at the top with all controls: - `<input type="text" id="fishName" placeholder="Name your fish">` - `<input type="color" id="colorPicker" value="#000000">` - `<input type="range" id="brushSlider" min="1" max="20" value="4">` - `<button id="eraserBtn">Eraser</button>` - `<button id="clearBtn">Clear</button>` - `<button id="spawnBtn">Spawn</button>` - `<canvas id="drawCanvas" width="800" height="500">`

Script tags (at the bottom, before `</body>`): - `storage.js`, `fish.js`, `painter.js`, `bubbles.js` — in that order

Step 1.3: style.css

Aquarium view elements:

SELECTOR	KEY PROPERTIES
`body`	`margin: 0; overflow: hidden; width: 100vw; height: 100vh; background: #0a1628;`
`#ocean`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 0;`
`#fishCanvas`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 1; pointer-events: none;`
`#baitBucket`	`position: fixed; z-index: 5; cursor: pointer;`

Floating draw button:

SELECTOR	KEY PROPERTIES
`#drawBtn`	`position: fixed; bottom: 30px; right: 30px; z-index: 21; width: 60px; height: 60px; border-radius: 50%; border: none; background: rgba(0,0,0,0.6); color: white; font-size: 28px; cursor: pointer;`

Drawing overlay:

SELECTOR	KEY PROPERTIES
`#drawOverlay`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 20; display: none; flex-direction: column; align-items: center; background: rgba(0,0,0,0.85);`
`#toolbar`	`display: flex; align-items: center; gap: 12px; padding: 12px; background: rgba(0,0,0,0.7); width: 100%; justify-content: center;`
`#drawCanvas`	`background: white; border-radius: 8px; cursor: crosshair; margin-top: 20px;`
`#fishName`	`padding: 6px 10px; border-radius: 6px; border: none; font-size: 14px;`

Step 1.4: Test It

1. Go to `chrome://extensions` in your browser
2. Enable **Developer mode** (toggle in the top right)
3. Click **Load unpacked** → select your TabTank folder (or refresh if already loaded)
4. Open a new tab

You should see: a dark blue screen with a round `+` button in the bottom-right corner. Clicking it toggles the drawing overlay. Clicking again closes it.

> **Testing Workflow (use this for every change going forward):** After every code change: go to `chrome://extensions`, click the refresh icon on your extension card, then open a new tab

> to see the changes.

Phase 2 — Drawing Tool (painter.js)

> **Status: MOSTLY COMPLETE**

Step 2.1: Get the Canvas Context

- Grab `#drawCanvas` with `document.getElementById`
- Call `canvas.getContext('2d')` — store the result as `ctx`
- Set initial drawing values:
- `ctx.lineCap = 'round'`
- `ctx.lineJoin = 'round'`
- `ctx.lineWidth = 4`
- `ctx.strokeStyle = '#000000'`

Step 2.2: Wire Up the Overlay Toggle

The draw button toggles the overlay open and closed. One click handler checks if the overlay is visible — if yes, close it and clear the canvas. If no, open it.

Step 2.3: Implement Freehand Drawing

Four event listeners on the drawing canvas:

EVENT	WHAT IT DOES
`mousedown`	Set `isDrawing = true`. Call `ctx.beginPath()`, then `ctx.moveTo(e.offsetX, e.offsetY)`.

EVENT	WHAT IT DOES
`mousemove`	If `isDrawing` is false, return early. Otherwise: `ctx.lineTo(e.offsetX, e.offsetY)`, then `ctx.stroke()`.
`mouseup`	Set `isDrawing = false`.
`mouseleave`	Set `isDrawing = false`.

> **Note:** The mouse event listeners need the `e` parameter for `e.offsetX` and `e.offsetY`. Other listeners (brush slider, color picker) can use the element variable directly instead.

Step 2.4: Color Picker

Listener on `colorPicker` that sets `ctx.strokeStyle` to the picker's current value. Also resets `globalCompositeOperation` to `'source-over'` (switches back from eraser mode).

Step 2.5: Brush Size

Listener on `brushSlider` that sets `ctx.lineWidth` to the slider's current value.

Step 2.6: Eraser

Click listener on `eraserBtn` that sets `ctx.globalCompositeOperation = 'destination-out'`. This erases to transparent. The color picker listener resets it back to `'source-over'` when a color is picked.

Step 2.7: Clear Button

Click listener on `clearBtn` that calls `ctx.clearRect(0, 0, canvas.width, canvas.height)` then fills with white:

```
ctx.fillStyle = 'white';
ctx.fillRect(0, 0, canvas.width, canvas.height);
```

> **Test checkpoint:** Open the overlay, draw with different colors and sizes, erase, clear. Everything should feel responsive on the large canvas.

Phase 3 — Fish Spawning & Swimming (fish.js)

Step 3.1: Set Up the Fish Canvas

- Grab `#fishCanvas` with `getElementById`
- Get its 2D context: `const fishCtx = fishCanvas.getContext('2d')`
- Set the canvas `width` and `height` **attributes** to `window.innerWidth` and `window.innerHeight`
- Add a `resize` event listener on `window` that updates these dimensions when the browser resizes

> **Important: Canvas Resolution vs CSS Size** CSS `width: 100%` stretches the canvas visually, but the canvas internal resolution is set by its `width` and `height` attributes. If you only set CSS size, your fish will be blurry. You must set both.

Step 3.2: Create the Fish Array

```
const fishArray = [];
```

`fishArray` holds all fish — both those currently dropping in and those swimming normally.

Each **fish object** has:

PROPERTY	TYPE	PURPOSE
`image`	Image	The drawn fish as an Image object
`name`	String	The name the user gave the fish

PROPERTY	TYPE	PURPOSE
`x`	Number	Current horizontal position
`y`	Number	Current vertical position
`destX`	Number	Destination X — the point the fish is swimming toward
`destY`	Number	Destination Y — the point the fish is swimming toward
`speed`	Number	How fast the fish swims (0.5–2.5, varies per fish)
`wobbleOffset`	Number	Random offset so fish don't bob in sync
`wobbleSpeed`	Number	How fast this fish wobbles (0.001–0.004, varies per fish)
`wobbleAmount`	Number	How much this fish wobbles (0.3–1.0, varies per fish)
`width`	Number	Rendered width on screen
`height`	Number	Rendered height on screen
`targetX`	Number or null	Bait target X (null = normal swimming)
`targetY`	Number or null	Bait target Y (null = normal swimming)
`opacity`	Number	Starts at 0, fades to 1 during drop-in
`spawning`	Boolean	`true` while fish is dropping in, `false` once swimming normally
`velocityY`	Number	Vertical velocity during drop-in (increases with gravity)
`targetSpawnY`	Number	The y position the fish is falling toward

How movement works: Instead of swimming in a straight horizontal line, each fish has a **destination point** (`destX` , `destY`) somewhere on screen. The fish swims toward that point. When it gets close, a new random destination is picked. This makes fish wander all over the tank naturally

— different directions, different heights, different speeds. Each fish also has its own wobble speed and amplitude, so no two fish move the same way.

How spawn drop-in works: When a fish is first created, `spawning` is `true`. The fish starts above the screen (`y = -100`) and falls downward with gravity (`velocityY` increases each frame). When it reaches `targetSpawnY`, it bounces (velocity reverses and shrinks). After a few bounces settle, `spawning` switches to `false` and the fish starts normal destination-based swimming.

Step 3.3: Wire Up the Spawn Button

Add a click listener on `#spawnBtn` that does the following, in order:

1. **Get the fish name:** `const name = fishName.value || 'Unnamed Fish'`
2. **Export the drawing:** `const dataURL = drawCanvas.toDataURL('image/png')`
3. **Create an Image object:** `const img = new Image()` then `img.src = dataURL`
4. **Wait for the image to load:** Everything below goes inside `img.onload = function() { ... }`
5. **Pick a random x position and target y:**
6. `x` : random between `100` and `fishCanvas.width - 200` (somewhere across the screen)
7. `targetSpawnY` : random between `50` and `fishCanvas.height - 150` (where the fish will settle)
8. `y` : `-100` (starts above the screen — offscreen)
9. **Pick a random first destination:**
10. `destX` : random between `50` and `fishCanvas.width - 100`
11. `destY` : random between `50` and `fishCanvas.height - 150`
12. **Pick random movement values:**
13. `speed` : random between `0.5` and `2.5` (each fish has its own speed)
14. `wobbleOffset` : random between `0` and `Math.PI * 2`
15. `wobbleSpeed` : random between `0.001` and `0.004` (how fast it bobs)
16. `wobbleAmount` : random between `0.3` and `1.0` (how much it bobs)
17. **Set dimensions:** Scale the drawing down to about 80–120px wide. Maintain aspect ratio.
18. **Create the fish object** with all properties including:
19. `targetX: null` , `targetY: null` (no bait target)
20. `opacity: 0` (starts invisible, fades in during drop)
21. `spawning: true` (drop-in animation active)

22. `velocityY: 0` (initial falling speed — gravity will increase this)
23. `targetSpawnY` (where the fish falls to)
24. **Push the fish directly into `fishArray`** — no separate animation array needed. The animation loop checks `fish.spawning` to decide whether to drop or swim.
25. **Close the overlay** and clear the draw canvas and name input.

Step 3.4: Build the Animation Loop

Create a function called `animate` that runs ~60 times per second. Every frame:

1. **Clear the entire fish canvas:** `fishCtx.clearRect(0, 0, fishCanvas.width, fishCanvas.height)`
2. **Loop through `fishArray`** and for each fish, check if it's spawning or swimming:

If `fish.spawning` is true — Drop-in animation:

Fade in as it falls: `javascript if (fish.opacity < 1) fish.opacity += 0.03;`

Apply gravity — velocity increases each frame, making the fish accelerate downward: `javascript fish.velocityY += 0.3; // gravity fish.y += fish.velocityY; // move down`

Check if fish reached its target y position: ```javascript if (fish.y >= fish.targetSpawnY) { fish.y = fish.targetSpawnY; // snap to target fish.velocityY *= -0.4; // bounce: reverse direction, lose energy

```
    // if bounce is tiny, stop bouncing and start swimming
    if (Math.abs(fish.velocityY) < 0.5) {
        fish.spawning = false;
        fish.opacity = 1;
        fish.y = fish.targetSpawnY;
    }
```

} `` The bounce works by reversing` velocityY `and multiplying by` 0.4 `— each bounce is smaller than the last. The fish overshoots down, bounces up a bit, falls again, bounces less, until the motion is tiny enough to stop. Then` spawning `flips to` false` and normal swimming begins.

If `fish.spawning` is false — Normal swimming:

Apply opacity (in case it's still fading in): `javascript fishCtx.globalAlpha = fish.opacity; // draw the fish... fishCtx.globalAlpha = 1; // reset for next fish`

Position update (destination-based swimming):

Calculate direction and distance to destination: `javascript const dx = fish.destX - fish.x; const dy = fish.destY - fish.y; const distance = Math.sqrt(dx * dx + dy * dy);`

Move toward destination: `javascript fish.x += (dx / distance) * fish.speed; fish.y += (dy / distance) * fish.speed;` `dx / distance` and `dy / distance` normalize the direction so the fish moves at a consistent speed regardless of how far away the target is.

Add wobble on top (so the fish doesn't move in a perfectly straight line): `javascript fish.y += Math.sin(Date.now() * fish.wobbleSpeed + fish.wobbleOffset) * fish.wobbleAmount;`

Pick new destination when fish arrives: `javascript if (distance < 20) { fish.destX = 50 + Math.random() * (fishCanvas.width - 100); fish.destY = 50 + Math.random() * (fishCanvas.height - 150); }` When the fish gets within 20px of its destination, a new random point is chosen. The fish turns and swims there. This creates natural wandering behavior — each fish roams all over the tank independently.

Draw the fish: (see Step 3.5 for the flip logic)

1. **Request the next frame:** `requestAnimationFrame(animate)`

Kick off the loop by calling `animate()` once when the page loads.

> **What is `requestAnimationFrame`?** The browser's built-in "call this function on the next screen refresh" method. Runs at ~60fps, automatically pauses when the tab isn't visible.

Step 3.5: The Flip Logic in Detail

When a fish swims left, the image needs to be mirrored horizontally. Since fish now swim toward destinations instead of a fixed horizontal direction, base the flip on **whether the destination is to the left or right** of the fish.

When swimming left (`fish.destX < fish.x`):

```
fishCtx.save();
fishCtx.translate(fish.x + fish.width, fish.y);
fishCtx.scale(-1, 1);
fishCtx.drawImage(fish.image, 0, 0, fish.width, fish.height);
fishCtx.restore();
```

When swimming right (`fish.destX >= fish.x`):

```
fishCtx.drawImage(fish.image, fish.x, fish.y, fish.width, fish.height);
```

`ctx.save()` and `ctx.restore()` ensure the transform only affects this one fish. The fish now faces the direction it's actually swimming — toward its current destination.

Step 3.6: Display Fish Names (Optional)

After drawing each fish, render its name below:

```
fishCtx.fillStyle = 'white';
fishCtx.font = '12px Arial';
fishCtx.textAlign = 'center';
fishCtx.fillText(fish.name, fish.x + fish.width / 2, fish.y + fish.height + 15);
```

> **Major test checkpoint:** Draw a fish, name it, click Spawn. The overlay should close. You should see the fish drop in from above the screen, accelerating downward, bouncing gently when it reaches its target position, then settling into normal swimming — wandering around the tank with its name, swimming toward random destinations, wobbling naturally, and flipping to face the direction it's heading. Spawn multiple fish — each should drop in at a different x position and wander independently with different speeds, wobble patterns, and destinations. **If this works, the hard part is done.**

Phase 4 — Ocean Backgrounds & Mode System

Step 4.1: Create Your Art

Open Procreate, Figma, or whatever design tool you prefer.

You need art for at least two modes:

Standard Ocean Mode

Resolution: 1920×1080 or larger, exported as PNG or JPEG.

For animated depth, split into 2–3 layers:

LAYER FILE	CONTENT	ANIMATION
`bg-deep.png`	Far background — deep water, gradient, distant elements	None (stays still)
`bg-foreground.png`	Close-up elements — coral, sand floor, seaweed, rocks	Slow horizontal sway
`bg-light.png`	Light rays / caustics (transparency as PNG)	Opacity pulse

Save into `assets/backgrounds/ocean/`.

Pixel Art Mode

Resolution: **320×180** — draw at this tiny resolution intentionally. The browser scales it up with hard pixel edges.

LAYER FILE	CONTENT	ANIMATION
`pixel-bg.png`	Full pixel art ocean scene at 320×180	None or subtle sway

Save into `assets/backgrounds/pixel/`.

Step 4.2: Add a Mode Toggle

Add a mode toggle button to your HTML:

```
<button id="modeToggle">Switch Mode</button>
```

Step 4.3: Add Image Layers to HTML

Inside `<div id="ocean">`, add `<img>` tags for each layer with classes like `ocean-layer-deep`, `ocean-layer-fg`, `ocean-layer-light`.

Step 4.4: Style the Layers

```
.ocean-layer {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    object-fit: cover;
}
```

Step 4.5: Pixel Mode CSS

```
body.pixel-mode #ocean img {
    image-rendering: pixelated;
}

body.pixel-mode #fishCanvas {
    image-rendering: pixelated;
}

body.pixel-mode #drawCanvas {
    image-rendering: pixelated;
}
```

Step 4.6: Mode Switching Logic (JavaScript)

The mode toggle button click does: 1. Toggle a class on `<body>` (e.g. `"pixel-mode"`) 2. Show/hide the correct background images 3. Optionally adjust the draw canvas resolution 4. Save the current mode to Chrome storage so it persists

Step 4.7: Animate with CSS Keyframes

Sway animation (foreground):

```
@keyframes sway {
    0%   { transform: translateX(-5px); }
    100% { transform: translateX(5px); }
}
```

```
.ocean-layer-fg {
    animation: sway 6s ease-in-out infinite alternate;
}
```

Light pulse (light rays):

```
@keyframes lightPulse {
    0%   { opacity: 0.25; }
    100% { opacity: 0.5; }
}

.ocean-layer-light {
    animation: lightPulse 4s ease-in-out infinite alternate;
}
```

> **Test checkpoint:** Your art fills the screen with subtle animation. Fish swim on top. Toggle between modes — backgrounds swap and pixel mode renders with hard pixel edges.

Phase 5 — Bait Feature

Step 5.1: Bait Mode Toggle

Variable: `let baitMode = false`

Click listener on `#baitBucket`: 1. Set `baitMode = true` 2. Change cursor on `#fishCanvas` to `crosshair` 3. Set `fishCanvas.style.pointerEvents = 'auto'`

Step 5.2: Ocean Canvas Click Handler

Click listener on `#fishCanvas`: 1. Guard: if `baitMode` is false, return 2. Get click coordinates from event 3. Find closest fish (loop through `fishArray`, calculate distances) 4. Set `closestFish.targetX` and `targetY` to click position 5. Reset bait mode, cursor, and pointer-events

Step 5.3: Update the Animation Loop

For each fish, before normal swimming, check if `fish.targetX !== null`:

If chasing bait: 1. Calculate angle: `Math.atan2(fish.targetY - fish.y, fish.targetX - fish.x)` 2. Rush speed: `6` 3. Move toward target using `Math.cos` / `Math.sin` 4. Flip based on target direction 5. If distance < 15px: clear target, pick a new random destination so the fish resumes wandering naturally

Else: Normal destination-based swimming (steer toward `destX` / `destY` with wobble).

> **Test checkpoint:** Spawn fish, click bait bucket, click ocean. Nearest fish darts to that spot then resumes swimming.

Optional polish: - Right-click anywhere drops bait (skip the bucket) - All nearby fish react within a radius - Shrinking circle visual at click point - Fish scales up ~10% briefly when reaching bait

---

Phase 6 — Persistence (storage.js)

Step 6.1: Save Function

`saveFish()` maps `fishArray` to serializable objects:

```
{
    dataURL:      fish.image.src,
    name:         fish.name,
    x:            fish.x,
    y:            fish.y,
    speedX:       fish.speedX,
    wobbleOffset: fish.wobbleOffset,
    width:        fish.width,
    height:       fish.height
}
```

Save with `chrome.storage.local.set({ fish: serializedArray })`.

Call after every spawn.

Step 6.2: Load Function

`loadFish()` : 1. `chrome.storage.local.get('fish', callback)` 2. Loop through saved fish, create `Image` objects, push into `fishArray` inside `img.onload`

Call on page load, before starting the animation loop.

Step 6.3: Save Mode Preference

```
chrome.storage.local.set({ mode: currentMode })
```

Load on startup and apply the correct body class.

Step 6.4: Delete a Fish

METHOD	IMPLEMENTATION	COMPLEXITY
"Clear all fish" button	Empty `fishArray`, call `chrome.storage.local.remove('fish')`	Easy — do this for v1
Right-click a fish	Find nearest fish, remove from array, save	Medium
Drag to trash icon	Skip for v1	Hard

> **Test checkpoint:** Spawn fish, close tab, open new tab. Fish reappear with names. Mode persists.

Phase 7 — Bubbles (bubbles.js) — Optional Polish

Step 7.1: Bubble Array

Each bubble: `x`, `y`, `radius` (2–6px), `speedY` (0.3–1.0), `drift`. Spawn 20–30 on page load.

Step 7.2: Animate Bubbles

APPROACH	HOW	PROS/CONS
Add to fish canvas loop	Draw translucent white circles with `fishCtx.arc()`	Simple, no extra elements.
Separate canvas	Third canvas between ocean and fish	Clean separation.
CSS-animated divs	Small round `<div>`s animated with `@keyframes`	Easiest. Recommended.

Phase 8 — Custom UI & Button Art

Step 8.1: Draw Your Buttons in Procreate

Design custom art for: draw button, toolbar buttons (eraser, clear, spawn), bait bucket, mode toggle. Export as PNG with transparency into `assets/ui/`. Draw pixel variants for pixel mode.

Step 8.2: Apply Custom Button Art with CSS

```
#drawBtn {
    border: none;
    background: url('../assets/ui/draw-btn.png') center/contain no-repeat;
}

body.pixel-mode #drawBtn {
    background-image: url('../assets/ui/pixel/draw-btn.png');
    image-rendering: pixelated;
}
```

Step 8.3: Hover & Active States

```
#drawBtn:hover { transform: scale(1.1); }
#drawBtn:active { transform: scale(0.95); }
```

Test checkpoint: All buttons display custom art. Hover/click feel responsive. Pixel mode swaps to pixel art buttons.

Layer Stack & Z-Index Order

```
┌──────────────────────────────────────────────┐
│ z-index: 0   Ocean background images (your art)│
│ z-index: 1   Fish canvas (transparent,fullscreen)│
│ z-index: 2   Bubbles (if using divs/extra canvas)│
│ z-index: 5   Bait bucket                      │
│ z-index: 20  Drawing overlay (when open)      │
│ z-index: 21  Floating draw button (always on top)│
└──────────────────────────────────────────────┘
```

Aquarium view (normal):

```
┌──────────────────────────────────────────────┐
│  bg-deep.png  (static)                        │
│    ↕ bg-foreground.png (slow sway ~5px)       │
│      ✨ bg-light.png (opacity pulse)          │
│                                               │
│        🐟 ← "Nemo" drops in from above        │
│        ↓  falls with gravity, bounces, settles │
│        🐟 "Nemo" starts wandering             │
│        🐟 "Bubbles" swimming on canvas        │
│        🫧 ambient bubbles floating up         │
│                                               │
│  🪣 bait bucket                          [+]  │
│                              (draw button) ↗  │
└──────────────────────────────────────────────┘
```

Drawing overlay (when open):

```
┌──────────────────────────────────────────┐
│ [Name: ___] [🎨] [━━] [Eraser] [Clear] [Spawn]│
│                                          │
│   ┌──────────────────────────────────┐   │
│   │                                  │   │
│   │                                  │   │
│   │         (large white drawing canvas)   │   │
│   │         draw your fish here      │   │
│   │                                  │   │
│   │                                  │   │
│   └──────────────────────────────────┘   │
│                                      [+] │
│                        (click to close) ↗ │
└──────────────────────────────────────────┘
```

Milestone Checklist

#	MILESTONE	HOW TO VERIFY
1	Extension loads, dark screen + floating draw button	New tab shows dark blue background with round `+` button
2	Draw button toggles fullscreen overlay open/closed	Click `+` → overlay appears. Click again → disappears.
3	Can draw on large canvas with colors/sizes/eraser/clear	Freehand drawing works in the overlay with all controls
4	Spawn triggers drop-in animation then fish swims	Draw, name, spawn → overlay closes, fish drops from above, bounces, starts swimming
5	Fish swim with name, wobble, and flip at edges	Multiple fish move independently with different speeds
6	Ocean art shows as background with CSS animation	Hand-drawn art fills screen with subtle sway and light pulse

#	MILESTONE	HOW TO VERIFY
7	Mode toggle switches between standard and pixel art	Backgrounds swap, pixel mode renders with hard pixel edges
8	Bait click makes nearest fish dart to cursor position	Fish rushes to click point then resumes normal swimming
9	Fish, names, and mode persist across new tabs	Close tab, open new one — everything preserved
10	Ambient bubbles float upward	Translucent circles rise from bottom of screen
11	Custom button art replaces browser defaults	Procreate art on all buttons with pixel variants

Quick Reference — Key APIs You'll Use

Canvas 2D Context

METHOD / PROPERTY	WHAT IT DOES
`getContext('2d')`	Gets the 2D drawing context from a canvas element
`beginPath()`	Starts a new drawing path
`moveTo(x, y)`	Moves the pen without drawing
`lineTo(x, y)`	Draws a line to (x, y)
`stroke()`	Renders the current path as a visible line
`clearRect(x, y, w, h)`	Erases a rectangular area
`drawImage(img, x, y, w, h)`	Draws an image onto the canvas

METHOD / PROPERTY	WHAT IT DOES
`arc(x, y, radius, start, end)`	Draws a circle/arc (used for bubbles)
`fill()`	Fills the current path with color
`save()` / `restore()`	Saves/restores the current transform state
`translate(x, y)`	Moves the canvas origin point
`scale(x, y)`	Scales drawing. `scale(-1, 1)` flips horizontally
`toDataURL('image/png')`	Exports canvas content as a base64 PNG string
`strokeStyle`	Sets the line/stroke color
`fillStyle`	Sets the fill color (for shapes and text)
`lineWidth`	Sets line thickness in pixels
`lineCap`	Sets line end shape (`'round'`)
`fillText(text, x, y)`	Draws text on the canvas
`globalCompositeOperation`	Controls blending (`'destination-out'` for eraser)
`globalAlpha`	Sets opacity for all drawing (0 = invisible, 1 = fully visible)

DOM & Events

METHOD	WHAT IT DOES
`document.getElementById('id')`	Finds an HTML element by its ID
`element.addEventListener('event', function)`	Runs code when event happens
`element.style.display = 'flex'`	Shows a hidden element
`element.style.display = 'none'`	Hides an element

METHOD	WHAT IT DOES
`element.classList.toggle('class')`	Adds/removes a CSS class

Math Functions for Animation

FUNCTION	WHAT IT DOES
`Math.sin(value)`	Returns -1 to 1 in a wave. Swimming wobble + bubble wobble.
`Math.cos(value)`	X-component of movement in a direction
`Math.atan2(dy, dx)`	Angle from one point to another. Bait chasing + bubble pop direction.
`Math.sqrt(x**2 + y**2)`	Distance between two points
`Math.random()`	Random 0–1. Used for spawn positions, bubble sizes, speeds.

Chrome Extension APIs

API	WHAT IT DOES
`chrome.storage.local.set({ key: value })`	Saves data persistently
`chrome.storage.local.get('key', callback)`	Retrieves saved data
`chrome.storage.local.remove('key')`	Deletes saved data

Browser Animation

API	WHAT IT DOES
`requestAnimationFrame(callback)`	Runs function on next screen refresh (~60fps)

API	WHAT IT DOES
`Date.now()`	Current timestamp in ms. Input for `Math.sin()` time-based animation.

CSS Pixel Art Mode

PROPERTY	WHAT IT DOES
`image-rendering: pixelated`	Scales images with hard pixel edges